

File Compression Tool using Huffman Coding

Test Case UT_01

Test Case ID	Name of Module	Test Case Description	Pre-conditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result (Pass/Fail)
UT_01	File Input & Selection	Verify that the system accepts a valid file selected by the user for compression.	Application is launched and the user has access to a valid input file.	1. Launch the application. 2. Click Select File. 3. Browse to sample.txt. 4. Select the file. 5. Confirm the selection.	File: sample.txt Type: TXT Size: 10 KB	The file is accepted and its name and size are displayed. No error message is shown.	To be filled after execution	

Test Case UT_02

Test Case ID	Name of Module	Test Case Description	Pre-conditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result (Pass/Fail)
UT_02	File Validation	Verify that the system handles an invalid or unavailable input file.	Application is running and the file-selection option is available.	1. Click Select File. 2. Enter/select an invalid or unavailable file. 3. Attempt to proceed with compression.	File: missing.txt Status: Does not exist	The system rejects the input and displays a clear error message such as 'File not found' or 'Invalid file'. Compression does not start.	To be filled after execution	

Test Case UT_03

Test Case ID	Name of Module	Test Case Description	Pre-conditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result (Pass/Fail)
UT_03	Frequency Analysis	Verify that the system correctly calculates the frequency of each character/byte.	A valid text file has been selected and can be read successfully.	1. Select the input file. 2. Start frequency analysis. 3. View the generated frequency table.	File content: AAAABBBCCD	The frequency table correctly reports A=4, B=3, C=2 and D=1.	To be filled after execution	

Test Case UT_04

Test Case ID	Name of Module	Test Case Description	Pre-conditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result (Pass/Fail)
UT_04	Huffman Tree Construction	Verify that the system constructs a valid Huffman tree from the calculated frequencies.	Frequency analysis has completed successfully.	1. Provide the frequency table to the Huffman module. 2. Insert frequency nodes into the priority queue/min-heap. 3. Repeatedly combine the two lowest-frequency nodes. 4. Complete the Huffman tree.	Frequencies: A=4, B=3, C=2, D=1	A valid Huffman tree is constructed and the root contains the total frequency of 10.	To be filled after execution	

Test Case UT_05

Test Case ID	Name of Module	Test Case Description	Pre-conditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result (Pass/Fail)
UT_05	Huffman Code Generation	Verify that unique prefix-free binary codes are generated from the Huffman tree.	A valid Huffman tree has been constructed.	1. Traverse the Huffman tree from the root. 2. Assign 0 to one branch and 1 to the other. 3. Generate codes for all leaf nodes. 4. Display the generated codes.	Input: AAAABBBCCD Frequencies: A=4, B=3, C=2, D=1	Every symbol receives a binary Huffman code. No generated code is a prefix of another generated code.	To be filled after execution	

Test Case UT_06

Test Case ID	Name of Module	Test Case Description	Pre-conditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result (Pass/Fail)
UT_06	Bitstream Encoding	Verify that the input file is encoded into a compressed binary bitstream using Huffman codes.	Huffman codes have been generated successfully.	1. Read the input data. 2. Replace each character/byte with its Huffman code. 3. Pack the resulting bits into bytes. 4. Write the encoded data to the compressed file.	Input: AAAABBBCCD Generated Huffman codes from TC UT_05	A compressed file is created containing the encoded bitstream and the information required for decoding.	To be filled after execution	

Test Case UT_07

Test Case ID	Name of Module	Test Case Description	Pre-conditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result (Pass/Fail)
UT_07	Compressed File Creation	Verify that a valid compressed output file is generated after Huffman encoding.	Valid input file, Huffman tree, and Huffman codes are available.	1. Select sample.txt. 2. Start compression. 3. Wait for encoding to complete. 4. Check the selected output location.	Input: sample.txt Output: sample.huff	sample.huff is created successfully at the selected location and is readable by the application.	To be filled after execution	

Test Case UT_08

Test Case ID	Name of Module	Test Case Description	Pre-conditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result (Pass/Fail)
UT_08	Huffman File Decompression	Verify that a valid Huffman-compressed file can be decoded and restored.	A valid compressed file generated by the application is available.	1. Select sample.huff. 2. Click Decompress. 3. Allow the system to read the Huffman information. 4. Decode the bitstream. 5. Save the restored file.	Compressed file: sample.huff	The compressed data is decoded successfully and the original file is recreated.	To be filled after execution	

Test Case UT_09

Test Case ID	Name of Module	Test Case Description	Pre-conditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result (Pass/Fail)
UT_09	Lossless Data Verification	Verify that decompression produces a file identical to the original input.	The original file has been compressed and subsequently decompressed successfully.	1. Locate the original sample.txt. 2. Locate the decompressed output. 3. Compare their contents byte-by-byte or using a checksum. 4. Display the verification result.	Original: sample.txt Decompressed: sample_decompressed.txt	The original and decompressed files are identical. The system reports successful integrity/lossless verification.	To be filled after execution	

Test Case UT_10

Test Case ID	Name of Module	Test Case Description	Pre-conditions	Test Steps	Test Data	Expected Results	Actual Result	Test Result (Pass/Fail)
UT_10	Compression Statistics	Verify that the system correctly calculates and displays compression statistics.	Compression has completed successfully and both file sizes are available.	1. Compress a valid input file. 2. Record the original file size. 3. Record the compressed file size. 4. Calculate compression ratio and percentage of space saved. 5. Display the statistics.	Original size: 100 KB Compressed size: 60 KB	The system displays the correct original size, compressed size, compression ratio (0.60), and space saved (40%).	To be filled after execution	